Project Name: AI Code Navigator (Backend) (week 1 and week 2)

Problem: API Was Completely Open and Unprotected

What Was the Problem?

The backend API had no security of any kind Anyone who knew the server URL could call any endpoint without a password or key This was a serious problem because:

- Anyone could call /api/upload-repo with any GitHub link and burn through our OpenAI API credits with no limit
- There was no rate limiting so someone could send thousands of search requests per second and crash the server
- The API accepted any input without checking it first so a badly formatted request would either crash the controller or cause an unexpected error deep in the code
- If OPENAI_API_KEY or MONGO_URI were missing from the .env file, the server would start normally but crash later when it actually tried to use them which made debugging very confusing
- Every response had a different format some returned { error: "..." } some returned { success: false } which made the API hard to use consistently

---

How Did We Fix It?

We added seven separate security improvements. Each one solves one specific problem.

- **API Key check** -> every request must now include an x-api-key header that matches the API_KEY in the .env file. If it does not match, the server returns 401 Unauthorized immediately without running any code
- **Rate limiting** ->upload endpoints are limited to 10 requests per 15 minutes per IP. Search is limited to 30 requests per minute. This prevents someone from spamming the API and draining OpenAI credits
- **CORS lockdown** -> the server now only accepts requests from the origins listed in ALLOWED_ORIGINS in the .env file instead of accepting requests from any website
- **Body size limit** -> request bodies larger than 1MB are rejected automatically. This prevents someone from sending a huge payload to crash the server

- **Input validation** -> every endpoint now checks the request body before running any business logic The code field must be a string, the repoUrl must match the exact GitHub URL pattern, the search limit must be a number between 1 and 20
- **Startup env check** -> when the server starts, it immediately checks that OPENAI_API_KEY, MONGO_URI, and API_KEY are all set. If any are missing it prints a clear error message and stops. The server never starts in a broken state
- **Request ID** ->every request now gets a unique ID attached to it. This ID appears in the response header so if something goes wrong a user can report the exact request ID and we can find it in the logs

---

What Files We Changed

1. middleware/auth.js (new file)

A single middleware function that reads the x-api-key header from the request and compares it to the API_KEY environment variable. If they do not match it returns 401 and stops the request If they match it calls next() and the request continues normally.

2. middleware/rateLimiter.js (new file)

Two separate rate limiters using the express-rate-limit package. uploadLimiter allows 10 requests per 15 minutes and is applied to /upload and /upload-repo. searchLimiter allows 30 requests per minute and is applied to /search. When the limit is hit it returns a clear error message instead of crashing

3. middleware/requestId.js (new file)

Generates a random UUID using Node's built-in crypto module and attaches it to every request as req.id. Also sends it back in the X-Request-Id response header so the caller can see which request had a problem.

4. validators/codeValidators.js (new file)

Uses the Joi library to define strict rules for each endpoint's request body. The validate() function wraps any schema and returns a middleware that checks the request before the controller runs. Unknown fields are stripped automatically. If validation fails it returns a 400 with a clear description of exactly what was wrong.

5. utils/envCheck.js (new file)

A small function called at the very top of index.js before anything else runs. It checks for three required environment variables and calls process.exit(1) with a clear error message if any are missing. The server never starts without them.

6. .env.example (new file)

A safe template showing every environment variable the project needs with placeholder values. It is safe to commit to git because it contains no real secrets. Anyone cloning the project copies this to .env and fills in their own values.

7. index.js

Added envCheck() at the very top so it runs before Express even starts. Added the requestId middleware. Changed cors() to only allow origins from the ALLOWED_ORIGINS environment variable. Added a 1MB body size limit to express.json().

8. routes/codeRoutes.js

Added auth, uploadLimiter, searchLimiter, and validate() to all four routes. Also added a file extension filter and 5MB per-file limit to the multer upload configuration so only code files are accepted.

9. models/code.js

Added timestamps: true so MongoDB automatically tracks createdAt and updatedAt on every document. Added required: true and an index on fileName for faster queries.

How We Tested It

We tested each security feature by trying to break it:

- Sent a request with no x-api-key header → received 401 Unauthorized
- Sent a request with a wrong API key → received 401 Unauthorized
- Sent a valid request with correct API key → request went through normally
- Sent a repoUrl with an invalid GitHub URL format → received 400 with the message "Must be a valid GitHub URL: https://github.com/owner/repo"
- Sent a search request with limit: 999 → Joi clamped it and returned 400
- Commented out MONGO_URI from .env and restarted the server → received "[startup] Missing required environment variables: MONGO_URI" and the server did not start
- Sent a request body larger than 1MB → received 413 Payload Too Large

---

Result

Before this week the API was completely open. Anyone with the server URL could call any endpoint, send any input, and there was no way to control or trace requests.

After this week the API requires authentication on every route, limits how fast requests can be sent, validates all inputs before they reach the code, checks the environment is correct before starting, and attaches a unique ID to every request for traceability. The server now behaves like a production API instead of a local development tool